ANN2: McCulloch-Pitts Neuron (ANDNOT) – Detailed Code & Viva Explanation

Cell 1: Import Library

Code:

```
import numpy as np
```

Explanation (what to say in viva):

In this cell, I am importing the numpy library.

Numpy is used for numerical operations and handling arrays.

Neural networks require mathematical calculations, so numpy is essential.

Cell 2: Initialize Weights and Threshold

Code:

```
w1 = 1
w2 = -1
theta = 1
```

Explanation (what to say in viva):

Here I am initializing weights and threshold for the MP neuron.

w1 is the weight for the first input and w2 is for the second input.

theta is the threshold value which decides neuron activation.

Weights control importance of inputs, and threshold controls firing condition.

Cell 3: Define MP Neuron Function

Code:

```
def mp_neuron(x1, x2):
```

Explanation (what to say in viva):

Here I am defining a function that represents the MP neuron.

This function will take two inputs and return the output.

Functions help reuse logic for different input combinations.

Cell 4: Compute Weighted Sum

Code:

```
yin = w1*x1 + w2*x2
```

Explanation (what to say in viva):

Here I am calculating the weighted sum of inputs.

Each input is multiplied with its corresponding weight.

Then both values are added to form the neuron input.

This simulates how biological neurons combine signals.

Cell 5: Apply Activation Function

Code:

```
if yin >= theta:
    return 1
else:
    return 0
```

Explanation (what to say in viva):

Here I am applying a step activation function.

If the weighted sum is greater than or equal to threshold, output is 1.

Otherwise, the output is 0.

This represents binary decision making of a neuron.

Cell 6: Define Input Combinations

Code:

```
inputs = [(0,0), (0,1), (1,0), (1,1)]
```

Explanation (what to say in viva):

These are all possible binary input combinations.

We test all combinations to verify the logic gate behavior.

This ensures the neuron works correctly for all cases.

Cell 7: Test the Model

Code:

```
for x1, x2 in inputs:
    print(x1, x2, mp_neuron(x1, x2))
```

Explanation (what to say in viva):

Here I am testing the MP neuron using all input combinations.

For each input pair, the function is called and output is printed.

This helps verify that the neuron implements ANDNOT logic.

The expected output is 0100.